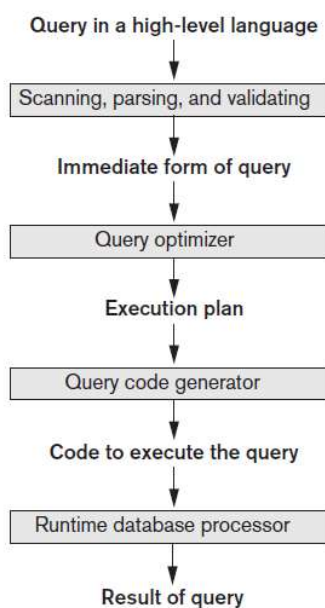


Chapter 11

Query Optimization

1

Query Processing



In SQLITE:
EXPLAIN QUERY PLAN
<query>

2

Complexity of Natural Join

assume index on key

Join On	$\max R_1(n) \bowtie R_2(m) $	Runtime complexity
Key(R_1) – Key(R_2)	$\min(m, n)$	$O(n \log m)$ (assume $n \leq m$)
Key(R_1) – NonKey(R_2)	m	$O(m \log n)$
NonKey(R_1) – Key(R_2)	n	$O(n \log m)$
NonKey(R_1) – NonKey(R_2)	mn	$O(nm)$

Theorem: In the natural join $R_1 \bowtie R_2 \bowtie \dots \bowtie R_k$, such that $R_i \bowtie R_{i+1}$ is on a unique attribute of R_{i+1} , we have:

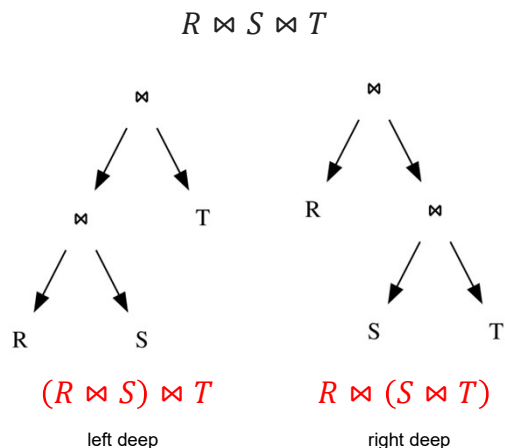
$$|R_1 \bowtie R_2 \bowtie \dots \bowtie R_k| \leq |R_1|$$

Conclusion: Since join is commutative and associative - for efficiency, it is best to order a sequence of natural joins in increasing order of table size.

3

Size Matters

Table	#Rows	Runtime
R	10	
S	10^4	
T	10^6	
$R \bowtie S$	10	$40 \log 10$
$S \bowtie T$	10^4	$60,000 \log 10$
$(R \bowtie S) \bowtie T$	10	$40 \log 10 + 60 \log 10$ $= 100 \log 10$
$R \bowtie (S \bowtie T)$	10	$60,000 \log 10 + 40 \log 10$ $= 60,040 \log 10$



4

Use Relational Algebra

List all employees who earn more than any employee in dept #5.

```
SELECT Lname, Fname
FROM EMPLOYEE
WHERE Salary > (SELECT MAX(Salary)
                FROM EMPLOYEE
                WHERE Dno=5);
```

Inner block

```
( SELECT MAX (Salary)
  FROM EMPLOYEE
 WHERE Dno=5 )
```

$$\mathcal{S}_{\text{MAX Salary}(\sigma_{\text{Dno}=5}(\text{EMPLOYEE}))}$$

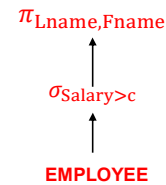
Outer block

```
SELECT Lname, Fname
FROM EMPLOYEE
WHERE Salary > c
```

$$\pi_{\text{Lname, Fname}}(\sigma_{\text{Salary} > c}(\text{EMPLOYEE}))$$

The query optimizer chooses an execution plan for each query block.

This is easy since the inner block is *uncorrelated* with the outer block.



5

Query Trees

List the names and titles of employees working on a project whose budget is at least \$250,000 and which employs more than two employees.

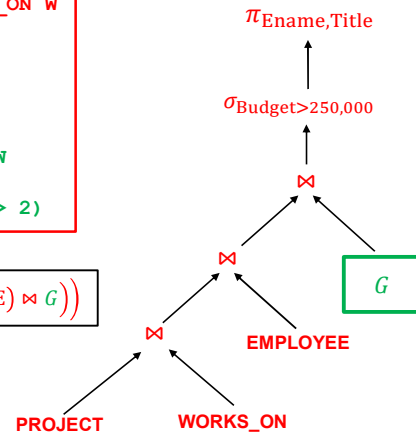
```
SELECT      E.Ename, E.Title
FROM        EMPLOYEE E, PROJECT P, WORKS_ON W
WHERE       P.Budget > 250000
           AND E.Ssn = W.Essn
           AND P.Pnumber = W.Pno
           AND P.Pnumber IN
               (SELECT W.Pno
                FROM   WORKS_ON W
                GROUP BY W.Pno
                HAVING COUNT(*) > 2)
```

How should we execute this query?

6

Query Trees

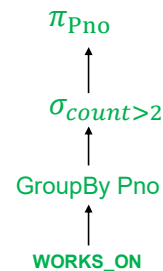
```
SELECT E.Ename, E.Title
FROM   EMPLOYEE E, PROJECT P, WORKS_ON W
WHERE  P.Budget > 250000
      AND E.Ssn = W.Essn
      AND P.Pnumber = W.Pno
      AND P.Pnumber IN
          (SELECT W.Pno
           FROM   WORKS_ON W
           GROUP BY W.Pno
           HAVING COUNT(*) > 2)
```

$$\pi_{Ename, Title} \left(\sigma_{Budget > 250,000} \left(((PROJECT \bowtie WORKS_ON) \bowtie EMPLOYEE) \bowtie G \right) \right)$$


7

Query Trees

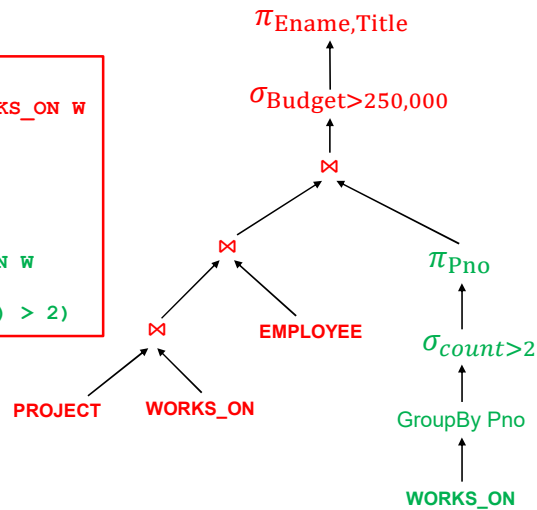
```
SELECT W.Pno
FROM   WORKS_ON W
GROUP BY W.Pno
HAVING COUNT(*) > 2
```

$$\pi_{Pno} (\sigma_{count > 2} Pno \mathcal{F} WORKS_ON)$$


8

Query Trees

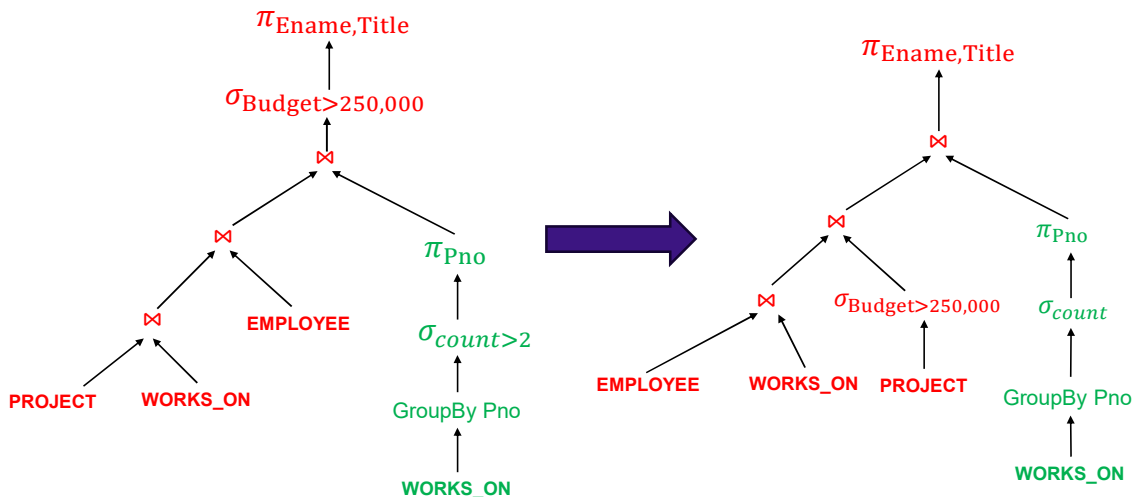
```
SELECT E.Name, E.Title
FROM   EMPLOYEE E, PROJECT P, WORKS_ON W
WHERE  P.Budget > 250000
      AND E.Ssn = W.Essn
      AND P.Pnumber = W.Pno
      AND P.Pnumber IN
        (SELECT W.Pno
         FROM   WORKS_ON W
         GROUP BY W.Pno
         HAVING COUNT(*) > 2)
```



$$\pi_{Ename, Title} \left(\sigma_{Budget > 250,000} \left(((PROJECT \bowtie WORKS_ON) \bowtie EMPLOYEE) \bowtie \pi_{Pno}(\sigma_{count > 2} Pno \mathcal{F} WORKS_ON) \right) \right)$$

9

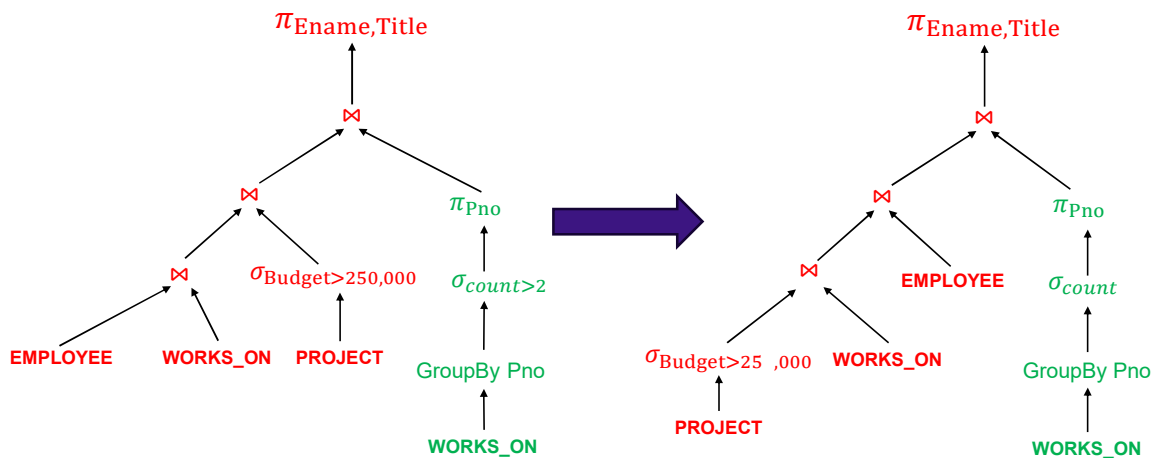
Alternative Query Tree



$$\pi_{Ename, Title} \left(\left((EMPLOYEE \bowtie WORKS_ON) \bowtie \sigma_{Budget > 250,000} PROJECT \right) \bowtie \pi_{Pno}(\sigma_{count > 2} Pno \mathcal{F} WORKS_ON) \right)$$

10

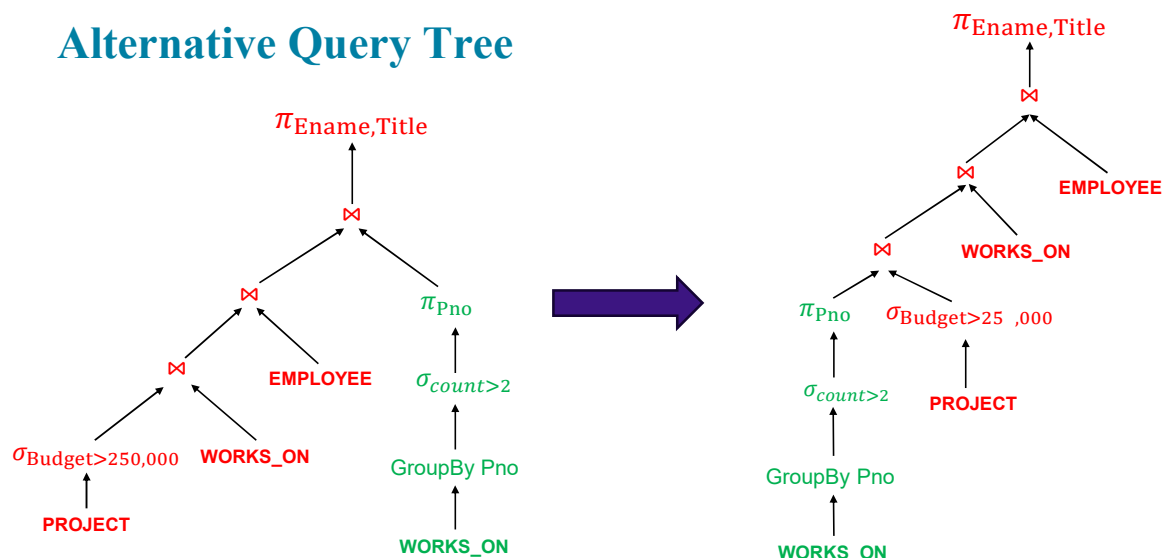
Alternative Query Tree



$$\pi_{Ename, Title} \left(\left(\left(\left(\sigma_{Budget > 250,000} PROJECT \right) \bowtie WORKS_ON \right) \bowtie EMPLOYEE \right) \bowtie \pi_{Pno} \left(\sigma_{count > 2} Pno \bowtie WORKS_ON \right) \right)$$

11

Alternative Query Tree

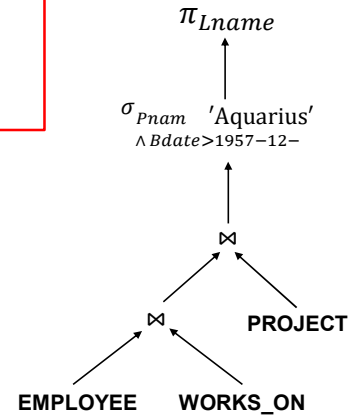


$$\pi_{Ename, Title} \left(\left(\left(\pi_{Pno} \left(\sigma_{count > 2} Pno \bowtie WORKS_ON \right) \right) \bowtie \sigma_{Budget > 250,000} PROJECT \right) \bowtie WORKS_ON \right) \bowtie EMPLOYEE$$

12

Query Transformation

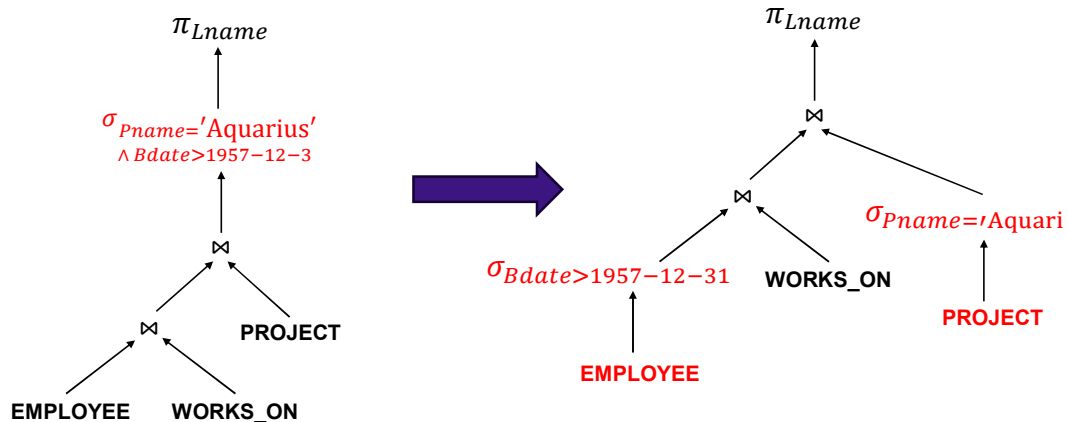
```
SELECT      E.Lname
FROM        EMPLOYEE E, PROJECT P, WORKS_ON W
WHERE       P.Pname = 'Aquarius'
           AND E.Ssn = W.Essn
           AND P.Pnumber = W.Pno
           AND P.Bdate > 1957-12-31;
```



13

Query Transformation

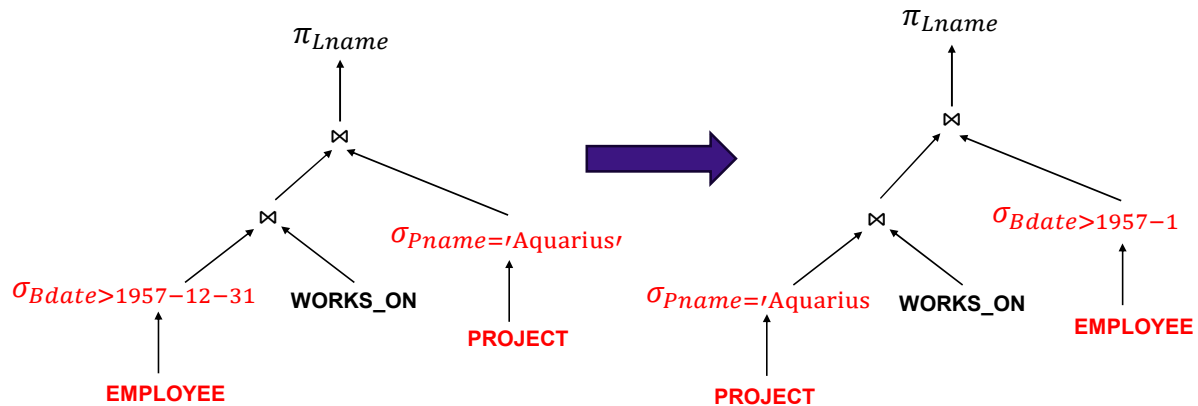
Move SELECT operations down the query tree to reduce table sizes.



14

Query Transformation

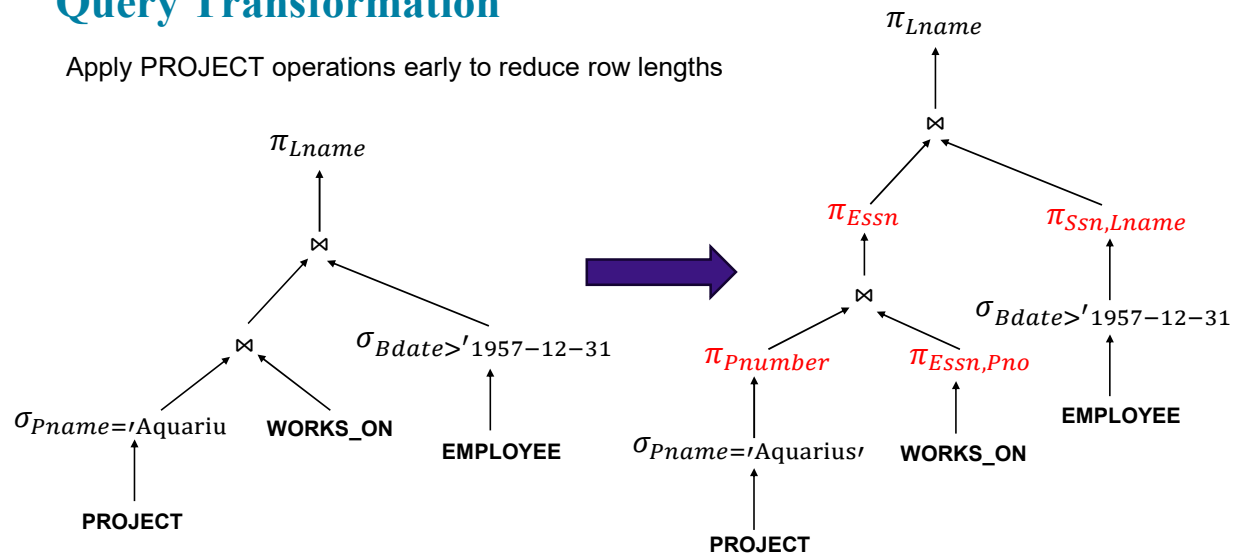
Apply the more restrictive SELECT operation first to reduce join complexity.



15

Query Transformation

Apply PROJECT operations early to reduce row lengths



16

Transformation Rules for Rational Algebra Expressions

Cascade of σ :

- A conjunctive selection condition can be broken up into a cascade (sequence) of individual selections

$$\sigma_{A \wedge B}(R) = \sigma_A(\sigma_B(R))$$

Commutativity of σ :

$$\sigma_B(\sigma_A(R)) = \sigma_A(\sigma_B(R))$$

Cascade of π :

- In a cascade (sequence) of projections, all but the last one can be ignored

$$\pi_A(\pi_B(R)) = \pi_A(R) \quad A \subseteq B$$

Commutativity of σ with π :

$$\sigma_A(\pi_B(R)) = \pi_B(\sigma_A(R))$$

17

Transformation Rules for Rational Algebra Expressions

Commutativity of $\bowtie$:

$$R \bowtie S = S \bowtie R$$

Associativity of $\bowtie$:

$$(R \bowtie S) \bowtie T = R \bowtie (S \bowtie T)$$

$$\bowtie R_i = \bowtie R_{perm(i)}$$

Distribution of σ over $\bowtie$:

$$\sigma_{C_1 \wedge C_2}(R \bowtie S) = \sigma_{C_1}(R) \bowtie \sigma_{C_2}(S)$$

if C_1 involves only Schema(R) and
 C_2 involves only Schema(S)

Distribution of π over $\bowtie$:

$$\pi_{A \cup B}(R \bowtie S) = \pi_{A \cup J}(R) \bowtie \pi_{B \cup J}(S)$$

if $A \subseteq \text{Schema}(R)$ and $B \subseteq \text{Schema}(S)$
and J are the join attributes

18

Heuristics for Query Optimization

SELECT → PROJECT → JOIN

Apply first the operations that reduce the size of intermediate results

- Perform SELECT and PROJECT operations as early as possible to reduce the number of tuples and attributes
- The SELECT and JOIN operations that are most restrictive should be executed first
- Use indexes to speed up JOIN where possible